

Recupero e implementazione di componenti ML per analisi dati in un contesto distribuito

Gabriele Stentella
984491

Relatore:
Prof. Claudio A. Ardagna

Correlatori:
Prof. Marco Anisetti
Dott. Antongiaco Polimeno

Nell'amministrazione dei moderni servizi accessibili via internet da un grande numero di utenti e di software integrati in sistemi ad alta automazione, va affrontata la gestione di un'elevata produzione di dati, fondamentali al fine di migliorare i servizi offerti, far crescere economicamente le aziende che li forniscono ed effettuare analisi sulla sicurezza. Questi dati sono caratterizzati da un grande volume che cresce velocemente nel tempo, spesso non sono strutturati e contengono anche informazioni che non hanno utilità, dunque per estrarre valore da tali dati, in letteratura chiamati *Big Data*, serve utilizzare delle tecniche di analisi non tradizionali, ovvero diverse da quelle che sono solitamente eseguite sui classici RDBMS.

Per svolgere analisi su insiemi di dati ad elevata dimensionalità, è necessario parallelizzare il carico computazionale in modo tale da dividere sia il dataset sia le operazioni da svolgere sui dati su diversi nodi di un *cluster*, siano essi macchine fisicamente distinte o nodi implementati in un ambiente virtualizzato. Con l'avvento dei *Big Data* sono quindi nati dei framework pensati per gestire sia la distribuzione di dati su più dispositivi, sia l'orchestrazione delle operazioni che vengono svolte su uno stesso insieme di dati, ma su nodi diversi; la prima architettura logica sviluppata è stata *Apache Hadoop*, un insieme di software *open-source* che consente di distribuire il carico computazionale con il modello *MapReduce* e distribuire i dati attraverso l'*Hadoop distributed file system* (HDFS). Per superare il processo sequenziale imposto dall'utilizzo di *MapReduce*, è stato sviluppato successivamente *Apache Spark* che consente di leggere i dati dalla memoria, eseguire operazioni e scrivere i risultati in un singolo step, inoltre ha introdotto la possibilità di riuso dei dati mantenuti in cache, caratteristica che consente una grande ottimizzazione di algoritmi che chiamano ripetutamente una funzione su uno stesso dataset, come quelli di *machine learning* (ML), infatti fra le API offerte da *Spark* si trova una libreria specificatamente dedicata al *machine learning* (MLlib), ampiamente utilizzata in questo lavoro di tesi. Un ulteriore vantaggio del framework *Spark* è quello di non avere un proprio sistema di archiviazione, ma di permettere l'utilizzo sia di sistemi esterni come HDFS o *Amazon S3*, sia di soluzioni personalizzate.

Gli algoritmi di ML possono essere utilizzati sia come strumento predittivo, ovvero per cercare di generalizzare le proprietà osservate in una collezione di dati, sia come strumento per effettuare il *data mining*, quindi per ricercare proprietà non note all'interno dei dati che si hanno a disposizione. In generale, l'utilizzo di intelligenza artificiale nell'analisi dati mira a trovare delle correlazioni fra le variabili presenti nella collezione di dati, con il fine di renderle evidenti ad un'analista oppure di utilizzarle per costruire un modello predittivo.

L'utilizzo sempre più pervasivo del *cloud computing* rende accessibile l'utilizzo di tecniche avanzate per l'analisi dati ad un numero crescente di realtà, e dunque è nata l'esigenza di avere algoritmi di *machine learning* che possano essere utilizzati come dei servizi, applicandoli a diverse collezioni di dati. L'obiettivo di questa tesi è quello di recuperare algoritmi di *machine learning* per l'analisi dati sviluppati come *Spark tasks* in linguaggio Java, e implementarne di nuovi seguendo un approccio più moderno, quindi utilizzando il linguaggio *Python* e la libreria *Tensorflow*; gli algoritmi vengono poi eseguiti in un contesto distribuito gestito dall'orchestratore *Kubernetes*.

Il lavoro di recupero degli algoritmi è stato portato avanti parallelamente allo studio e all'implementazione di nuovi algoritmi, e il lavoro di tesi può essere riassunto dalle fasi seguenti.

- Analisi degli algoritmi esistenti. È stata effettuata un'analisi dei diversi algoritmi già implementati al

fine di individuare quelli più interessanti per quanto riguarda l'analisi dei dati, in seguito ogni algoritmo selezionato è stato analizzato in dettaglio per individuarne le dipendenze e le versioni delle diverse tecnologie utilizzate.

- Aggiornamento e riorganizzazione degli algoritmi. Gli algoritmi selezionati sono stati aggiornati per utilizzare la versione più moderna di *Spark*; una libreria non più mantenuta, ma utilizzata dagli algoritmi è stata altresì aggiornata per renderla compatibile, inoltre la struttura dei progetti dei singoli task è stata ripensata in modo tale da poter costruire i diversi *file archivio* (jar), rispettando le eventuali dipendenze presenti fra alcuni task.
- Implementazione di un algoritmo di test in Tensorflow. È stato sviluppato un modello predittivo di *machine learning* in *Python* utilizzando la libreria *Tensorflow*.
- Esecuzione distribuita degli algoritmi. I task sono stati modificati in modo tale da consentirne l'esecuzione su un cluster *Kubernetes*, e in seguito sono stati eseguiti su un cluster appositamente creato per il testing, virtualizzato su una singola macchina, successivamente è stato implementato un secondo cluster di testing in cloud utilizzando le tecnologie *Amazon EMR on EKS* e *Amazon S3*.
- Test di performance dell'esecuzione distribuita. Sono stati analizzati i dati di log dei task *Spark* e quelli dell'allenamento del modello sviluppato con *Tensorflow*, al fine di dimostrare l'effettiva distribuzione del carico computazionale e dunque i vantaggi dell'esecuzione in contesto distribuito.
- Esecuzione sull'infrastruttura MUSA. Il *deployment* finale è stato eseguito sfruttando l'ambiente distribuito gestito con *Kubernetes*, all'interno dell'infrastruttura sviluppata nell'ambito del progetto MUSA, finanziato con i fondi del PNRR da parte del Ministero dell'Università e della Ricerca.

Gli sviluppi futuri per il lavoro cominciato con questa tesi sono molteplici: in primo luogo è possibile utilizzare un sistema di *scheduling* dei flussi di lavoro come *Apache Oozie* o *Airflow* per creare delle *pipeline* di task in modo tale da creare un'infrastruttura idonea ad analisi dati più complessa, inoltre l'approccio proposto per l'implementazione di nuovi algoritmi utilizzando *Python* e *Tensorflow* è scalabile e quindi resta aperta la possibilità di sviluppare nuovi modelli per l'analisi dati o rendere più efficienti algoritmi già implementati o noti in letteratura.